Rīgas Tehniskā Universitāte

Datorzinātņu un Informācijas Tehnoloģiju fakultāte

Priekšmets: Projektēšanas laboratorija

Universitātes telpu gaisa kvalitātes pārraudzības sistēma

Komanda:

Katrīna Anete Goba

Maija Rudzīte

Agnese Krūmiņa

Konrāds Rudzītis

Raivis Amoliņš

Oskars Grauzis

Rīga, 2023/2024 mācību gads

Saturs

1 Ievads _↑_ ... 3
2 Līdzīgo risinājumu pārskats _↑_ ... 5
3 Tehniskais risinājums _↑_ ... 6
3.1 Prasības ... 6
3.2 Algoritms _↑_ ... 7
3.3 Konceptu modelis _↑_ ... 11
3.4 Tehnoloģiju steks _↑_ ... 12
3.5 Programmatūras apraksts ... 12
3.6 Programmatūras apraksts ... 13
3.6.1 Web GUI lietotāja ... 13
3.6.2 Mērījumu ievākšana ... 16
3.6.3 Mērījumu arhivēšanas un dzēšanas uzdevums ... 16
3.6.4 mērījumu ievākšanas saglabātāji (.py skripti uz 2 portiem), ... 17
4 Novērtējums _↑_ ... 18
5 Secinājumi _↑_ ... 20

1 Ievads

Mūsu komanda bija izvēlējusies veidot projektu Universitātes gaisa kvalitātes pārvaldības sistēma. Pēc sarunas ar "pasūtītāju" iezīmējām galvenos lietotāja stāstus. Tākā viens no stāstiem bija saslēgšana, tad realizējām 3 dažādu interfeisu sensorus, un vienu no tiem kalibrējām paši.
Lai gan mūsu komandā bija pamanāmi cilvēki ar ideālistiskiem skatījumiem, tomēr jāņem vērā, ka nav dzirdēts par plašai publikai brīvi pieejamiem nemirstības risinājumiem, kas nozīmē, ka projekta izstrādei ir tikai neliels laika resurss, kas nozīmē, ka jākoncentrējas uz MVP sasniegšanu, nevis jāiedziļinās dažādos interesantās blakusnodarbēs kā absolūtās drošības meklējumi, neskaitāmu interfeisu izmēģināšana un tamlīdzīgi.

Rezultātā ieguvām mūsuprāt ievērojami dzīvotspējīgu, pietiekami ērti lietojamu un visnotaļ viegli mērogojamu sistēmas iestrādni. Kurai nav papildus atkarība no kādas attālinātas korporācijas lēmumu mainības vai nemainības. Tātad pielietojama arī attālākos civilizācijas punktos vai sasniedzot briedumu arī citur kosmosā.

Nosakot vadlīnijas, nolēmām, ka sekošanu hype un dažādiem modes zīmoliem šeit neprioritizēsim, bet tieši pretēji - vērsīsim skatus uz pietiekami ilgi esošām sistēmām ar maksimāli iespējamo koda caurredzamību, tajā pašā laikā kā jau ļoti limitētu resursu sistēmās, neaizmirstot par resursu limitiem.

Tāpēc izvēlējāmies python, un tikai divos gadījumos: webkameras blakus mini projektā un industriālā 4-20mA standartsensora gadījumā, lietojām c++. Pirmajā gadījumā jo kamerai bija kritiski nepieciešama operatīvā atmiņa, bet sensora gadījumā - jo kontrolierim ar augstākas izšķirtspējas ADC micropython netiek uzturēts, kas nozīmē ka liekot savu portu, ļoti ievērojami daudz laika aizietu mappinga, testēšanas un citos uzdevumos, kas acīmredzami nebūtu laika resursa daudzuma ziņā samērīgi ar projekta kopējo laika resursu.

Mikrokontrolieru kods ir asinhronu uzdevumu struktūrā veidots, un pat zemākā gala kontrolieris (esp32) to spēja izpildīt, pat esot uz operatīvās atmiņas pārpildes robežas. (jāpiezīmē, ka ja kodā atstāja 5 sensorus, jau pie izpildes bija kļūda ka nepietiek RAM. Saīsinot kodu uz 2 sensoriem viss atkal strādāja).
Arī API endpointu kods ir tajā pašā asyncio satvarā veidots. Arhivators turpretī ir sinhrons, jo tas ir vienlaikus viens uzdevums, tāpēc asinhromums ir lieks. Savukārt lietotāja saskarne ir salikts projekts, sastāvošs no Grafana, kas rakstīta go valodā un pielieto react un angular javascript satvarus un no Django satvarā veidotas CRUD un maršrutēšanas lietotnes, kas pielieto DataTables javascript satvaru sarakstu kārtošanai un meklēšanai, jo Django diemžēl modeļus atlasīja ievērojami lēni.

Darba mērķi bija vairāki,

- Izveidot mājaslapas saslēgumu ar sensoru datiem
- Parādīt datus reāllaika, vēlams vizuāli viegli uztveramā formā un iezīmējot kritiskās gāzu vērtības
- Dot iespēju regulēt datu ievākšanas biežumu, arhivēšanas biežumu un dzēšanas biežumu
- Dot iespēju veidot sensoru aprakstošo datu (metadatu) datubāzi un tai piesaistītās CRUD operācijas

Novērtēšanas mērķi ir pārliecināties, ka ir funkcionāls MVP pēc iespējas vairākos lietotāja stāstos nosauko prasību aspektos, kā arī vai ir vai arī ja nav, tad vai ir samērā vienkārši un ātri realizējamas drošības prasības.

2 Līdzīgo risinājumu pārskats

1. CISCO sensors Meraki MT14 (https://documentation.meraki.com/MT/MT_Datasheets/MT14_Datasheet_-_Indoor_Air_Quality_Monitor)
2. CO2 sensors DP-353 (https://www.amazon.com/Detector-Monitor-Display-Humidity-Temperature/dp/B0BMGF21RT?th=1)
3. ELRO Connects SF500CO2 CO2 Meter Kit (https://www.elro.eu/en/elro-connects-smart-wifi-co2-meter-kit-complete-set-with-linkable-air-quality-meter-k2-connector-sf500co2)
4. Disruptive Technologies Sensor Starter Kit (https://www.disruptive-technologies.com/products/wireless-co2-sensor)
5. WiFi Smart 3 In 1 Carbon Dioxide Air Quality Monitor Detector Temperature and Humidity Sensor (https://moeshouse.com/products/wifi-3-in-1-co2-monitor TUYA smart)
6. Tech stack alternatīvas:
    a. openHAB un kontrolieri kā mqqt klienti
    b. freeRTOS uz konroliera (https://github.com/aws-iot-builder-tools/freertos-pi-pico)

	1	2	3	4	5
CO2	+	+	+	+	+
Temp	+	+	+	+	+
Mitrums	+	+	+	+	+
WiFi	+	+	+	-/868Mhz	+
Pieejams vieglās valodas datu novērtējums	+	+	+	+	+/-
Mērījumu vēsture	+/-	-	+/-	+	-
Iespēja arhivēt datus	-	-	-	-	-
Iespēja regulēt biežumu	-	-	-	+	-
Iespēja modificēt	+/-	-	-	-	-
Cena (eur) bez piegādes	195,24	56.99	149.00	669,00*	77.70

* ar piegādi

+/- -samērā viegli pieejams, taču nepieciešama papildus aparatūra vai konfigurācija

3 Tehniskais risinājums

3.1 Prasības

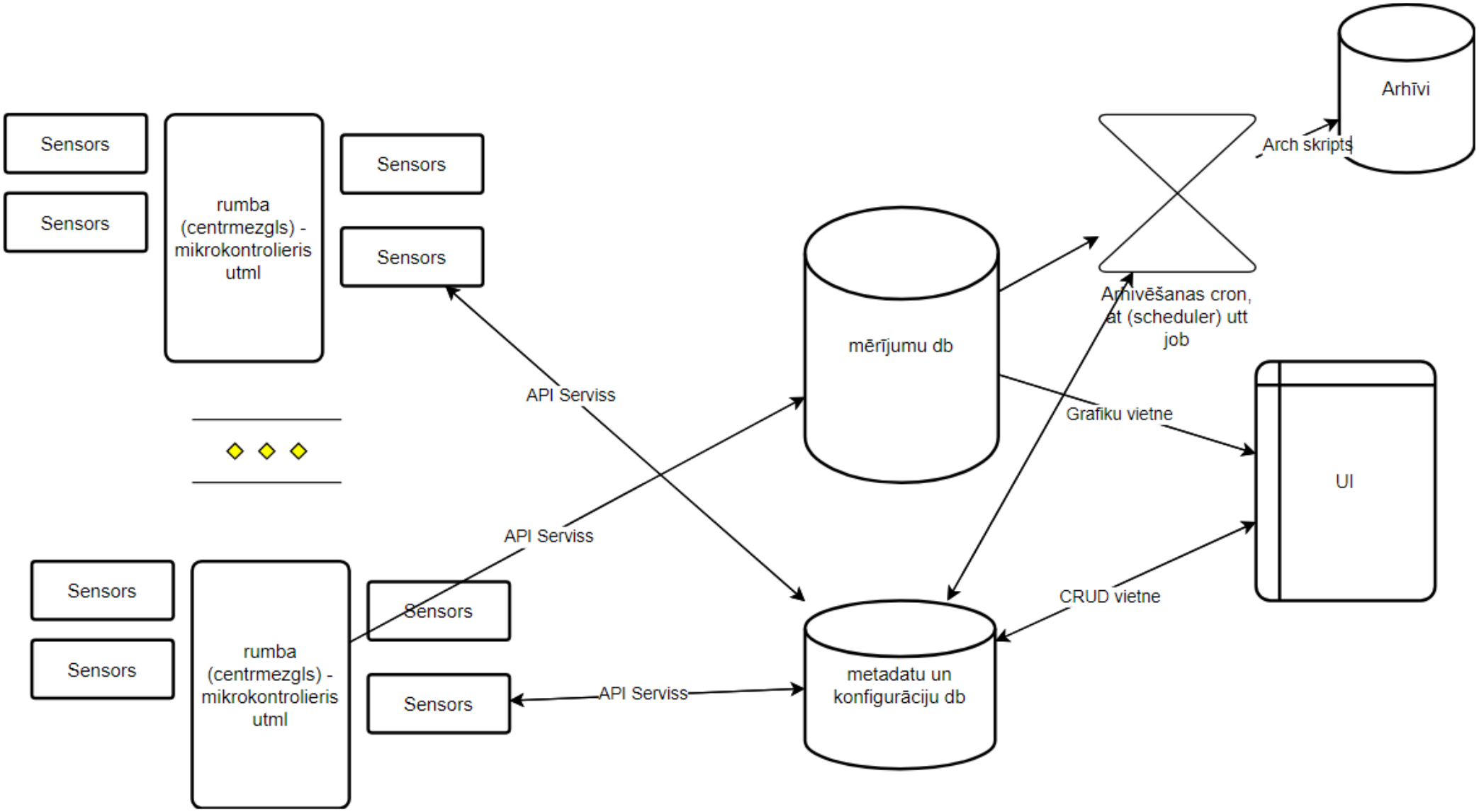
Nr.	Lietotāju stāsts <lietotājs> vēlas <sasniegt mērķi>, jo	Prioritāte	Izpilde
1.	Pasūtītājs vēlas savienotus sensorus ar rādītāju paneli, jo šī brīža risinājums nav pietiekami ērts.	A-> Must haves	J
2.	Pasūtītājs vēlas iespēju regulēt datu apmaiņas biežumu, jo ir jāatrod balanss starp biežumu, pārskatāmību un enerģijas patēriņu.	A-> Must haves	J
3.	Pasūtītājs vēlas rādītāju panelī redzēt sensora rādījuma interpretāciju ar prasībām publiskajā telpā, jo sensoru cipari nav intuitīvi / ātri uztverami	A-> Must haves	J
4.	Pasūtītājs vēlas rādītāju panelī iespēju regulēt glabāšanas laikus, jo laika gaitā mēdz mainīties pat likumdošana, kur nu vēl viedokļi par glabāšanas ilgumiem.	B-> Should haves	J
5.	Pasūtītājs vēlas trīs glabāšanas stratēģijas (db/arhīvs/dzēš), jo ir jāņem vērā veiktspēja arī pie bieža datu apmaiņas intervāla	B-> Should haves	J
6.	Pasūtītājs vēlas redzēt datus grafiski, jo cilvēkam dabiskāk un ātrāk uztverams ir zīmējums nekā apjomīgi skaitliski dati.	B-> Should haves	J
7.	Pasūtītājs vēlas iespēju meklēt/filtrēt sensorus atbilstoši atrašanās vietai, jo daudzu sensoru vidē nereti interesē konkrēts vai konkrēta to apakškopa.	B-> Should haves	J
8.	Pasūtītājs vēlas filtru sasaisti ar reālo telpas vietu, jo vizuāli ir ātrāk orientēties nekā ar simboliem.	C-> Could haves	N

3.2 Algoritms ↑

Sistēmu var ieadalīt 4 daļās:

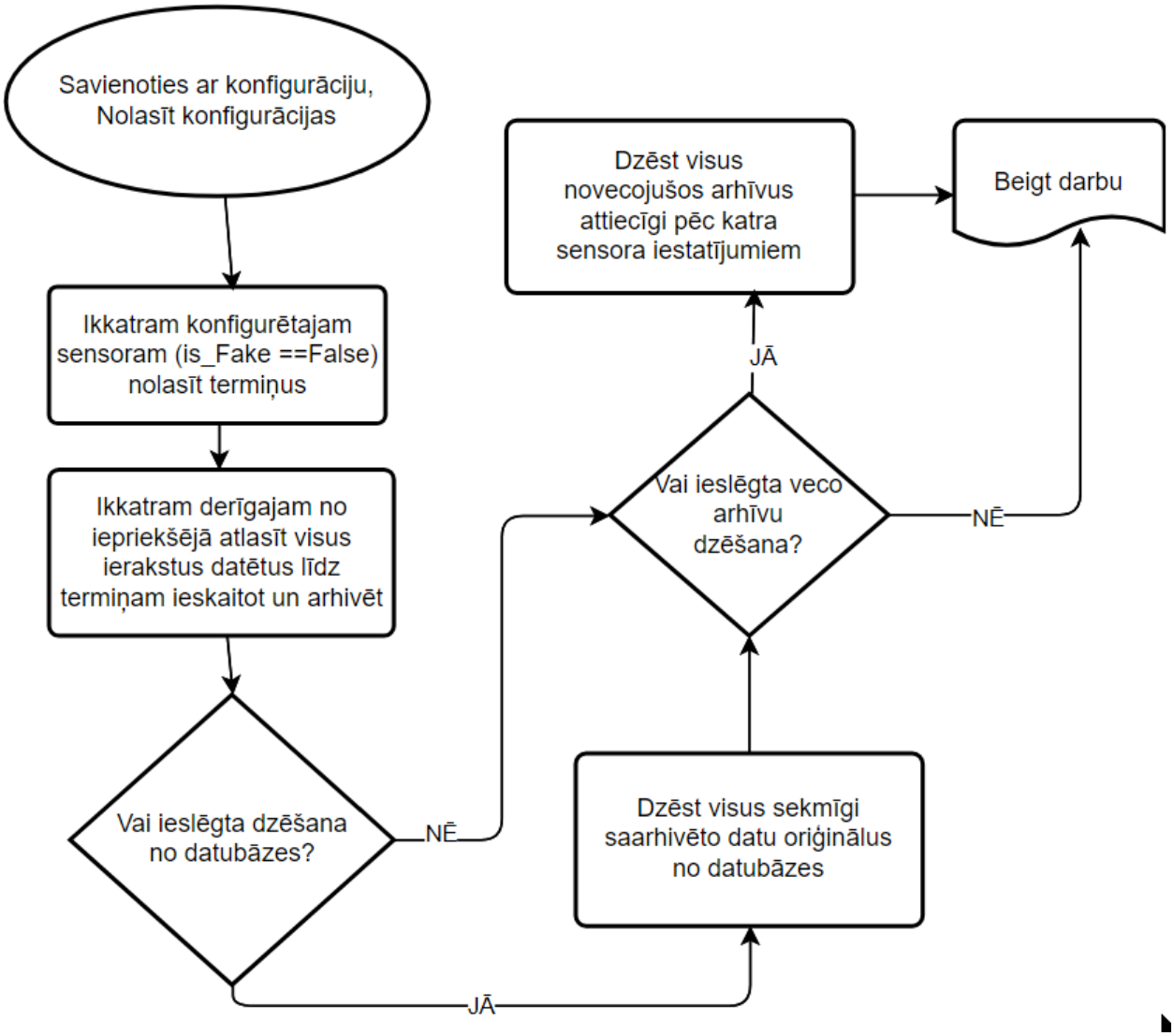
- mērījumu ievākšana (centrmezgli, mikrokontrolieri),
- mērījumu arhivēšanas un dzēšanas chron uzdevums,
- mērījumu ievākšanas saglabātāji (.py skripti uz 2 portiem),
- web GUI lietotājam (Go valodā rakstītā Grafana un CRUD operāciju Django).

Sistēmas kopskats: ↑

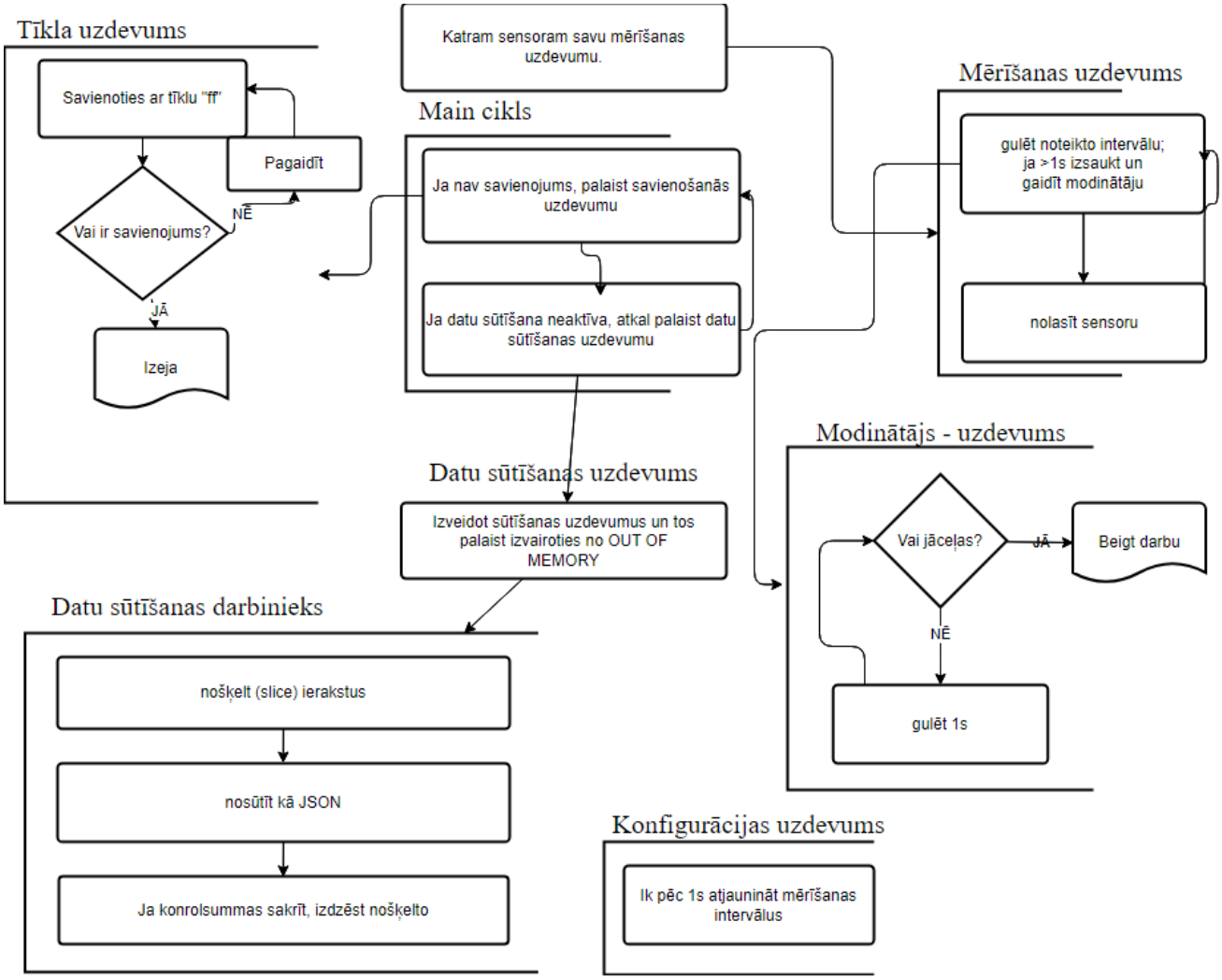


Arhivatora algoritms:

Dzēšana notiek ar panda dataframes palīdzību, jo arhivējot no sql tiek pārveidots csv un ar gzip saspiests.



Centrmezgla (mikrokontroliera) algoritms (katra grupa ir atsevišķs async tasks): ↑



Kalibrēšana 2B sensoram:

Bez nobīdes būtu: $V_{kal} = \frac{I_{mēr} * V_{ref}}{I_{rangs}}$

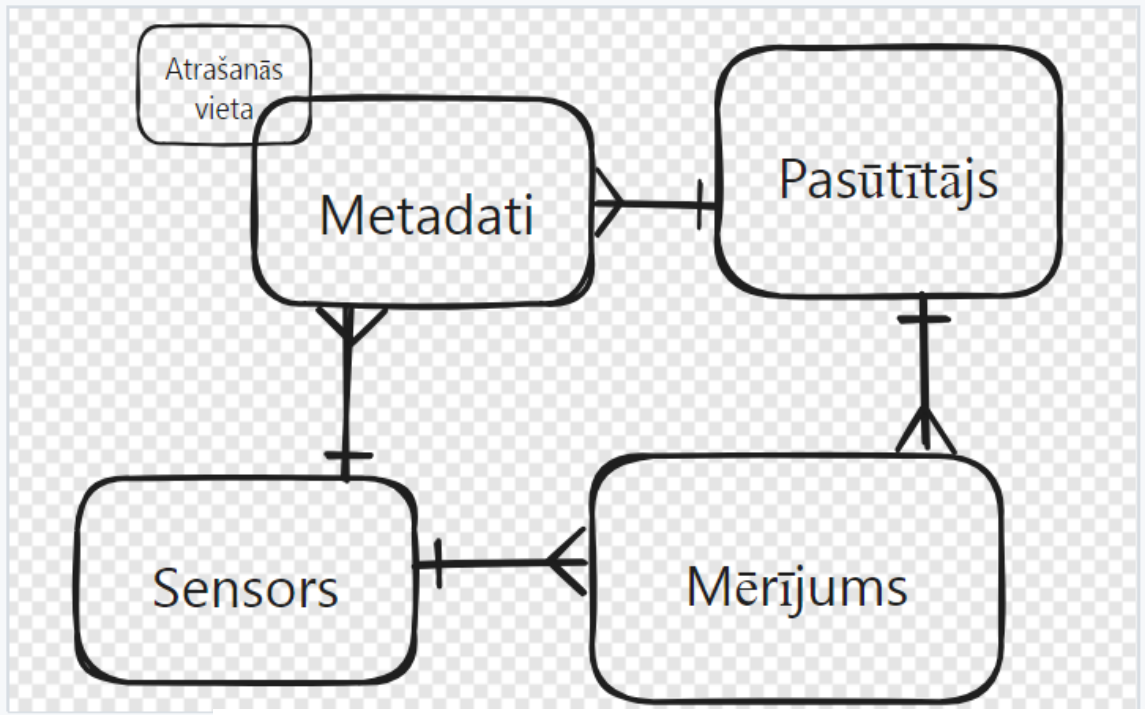
Ieejas ķēdes pretestību, kapacitāti,
Tāpēc "kalibrēju" ar ķēdē ieslēgtu s

Atsk nulli ignorēšu, aprēķinu pret V_{ref} teorētisko vism
linearitāte pat ar pārveidotāju kā starpnieku. Ja ne, at

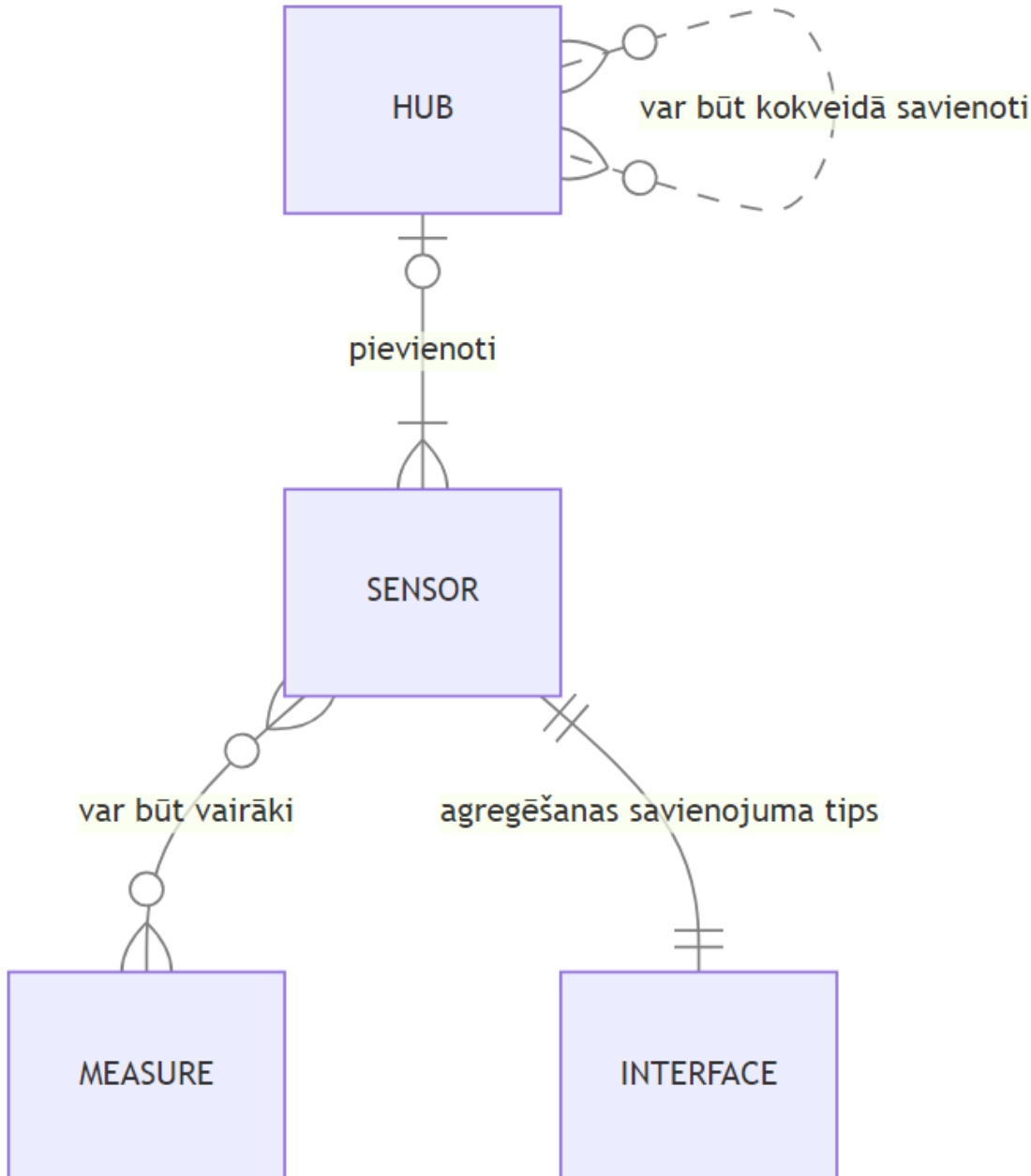
$$\Longrightarrow V_{m-kal} = \frac{I_{mēr} * 3300}{20} \Longrightarrow \qquad V_{m-kal} = I_{mēr} * 165$$

3.3 Konceptu modelis ↑

Sistēmas:



Metadatu:



3.4 Tehnoloģiju steks

Satvari: Django/Micropython (mikrokontrolierim)/Grafana (grafiki)

Programmēšanas valoda: Python

Datu bāzes: MySQL/SQLite

Tīmekļa serveri: Apache (produkcijas)/Python Django (izstrādes)

Operētājsistēma: Ubuntu server

Virtualizācija: VirtualBox

3.6 Programmatūras apraksts

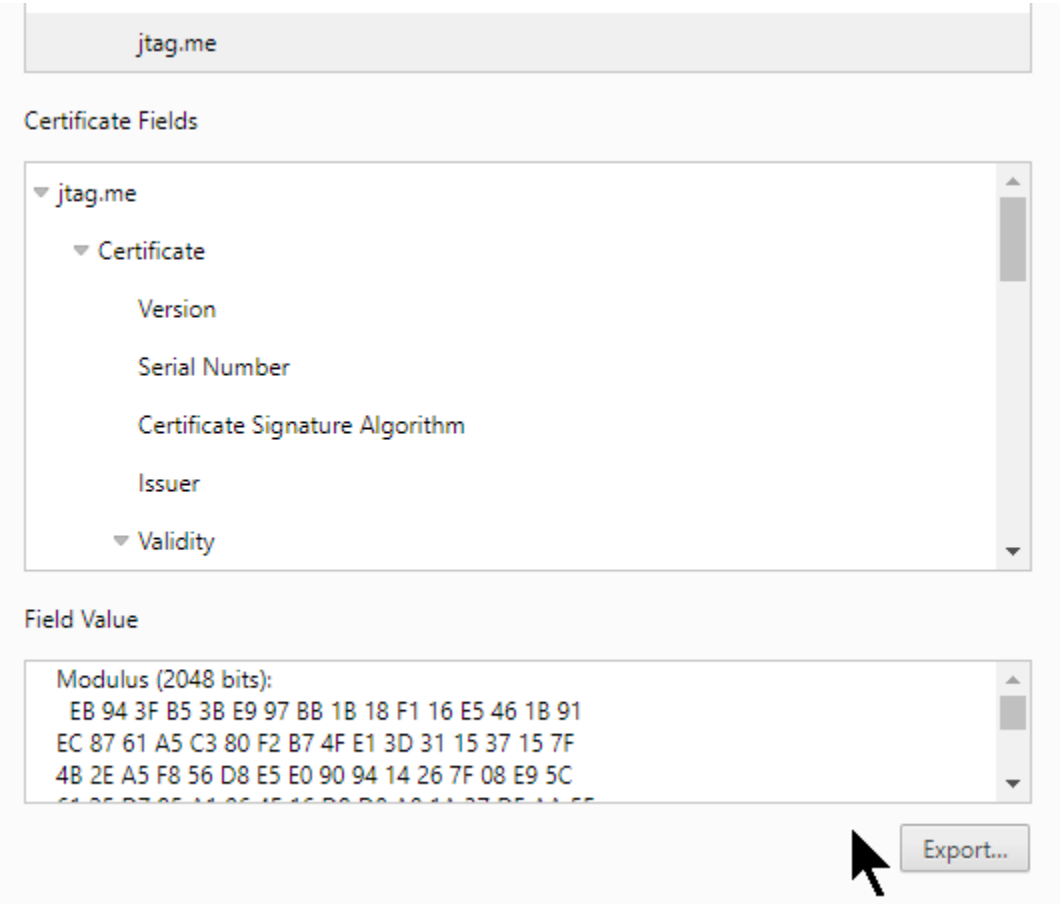
Programmatūru var iedalīt četrās apakšsistēmās:

- mērījumu ievākšana (centrmezgli, mikrokontrolieri),
- mērījumu arhivēšanas un dzēšanas chron uzdevums,
- mērījumu ievākšanas saglabātāji (.py skripti uz 2 portiem),
- web GUI lietotājam (Go valodā rakstītā Grafana un CRUD operāciju Django)

3.6.1 Web GUI lietotāja

3.6.1.1 Grafana

Web GUI lietotājam ir uz prezentācijas brīdi nohostēts un piereģistrēts kā vietne https://dj.jtag.me/ . Mēģinot atvērt bez šifrēšanas, proti, ar http priekšā web severis ir nokonfigurēts atbildēt ar 302 moved permanently un pāradresēt uz ar ssl 2048 bitu RSA sertifikātu šifrēto datu plūsmu.

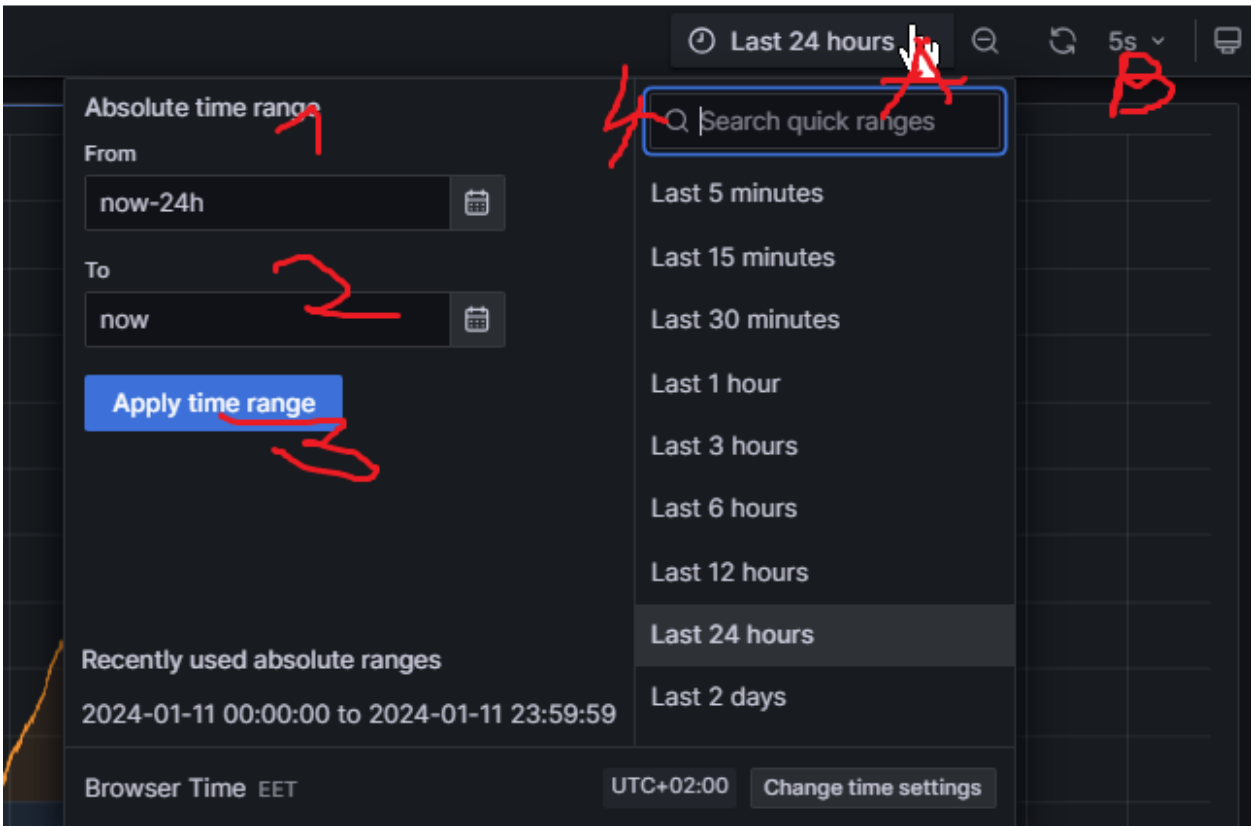


Web interfeiss domāts publiskai pieejai, tomēr demonstrācijas nolūkos it visi pieminētie galapunkti ir ar triviālu autentificēšanos. Triviālu jo esam ieslēguši iespēju reģistrēties ar jebkādu (izņemot tukšu) lietotājvārdu un paroli, nekas cits netiek prasīts.

Šajā mājaslapas skatā katrs grafiks aizņem pilnu lietotāja ekrāna platumu un augstumu, un uz nākamo ekrānu iespējams pāriet vai nu ritinot, vai velkot ar vienu pirkstu skārienjūtīgās ierīces ekrānu. Ekrāni tiek redzami atdalīti ar horizontālo līniju (hr html valodas iezīme (tag)). Ekrānus nodrošina grafanas jaunākā versija, kura darbojas docker konteinerā un pieņem savienojumus tikai un vienīgi no localhost. Grafana ir sarakstīta go valodā, bet tās frontendu nodrošināja angular (1. grafikam) un react (p4 grafikam). Būtisku atšķirību ielādes laikos angular un react nepamanījām, lai gan testējām tikai uz samērā mūsdienīgām ierīcēm un sistēmām. Toties pamanījām ka ir plašas optimizācijas iespējas, piemēram, izņemt no typescript ārā lokalizācijas valodās, kas samazinātu gala javascript izmērus.



Grafika apskate ir samērā intuitīva, nospiežot labā augšējā daļā (A) ir iespējams izvēlēties laiku no (1) līdz (2) un apstiprināt izvēli (3). Nospiežot B iespējams izvēlēties atjaunināšanas laiku. Jāpiezīmē ka tas ir cik bieži grafiks apjautā datubāzi un nav bijis paredzēts kā sensora datu ievākšanas biežuma regulators.



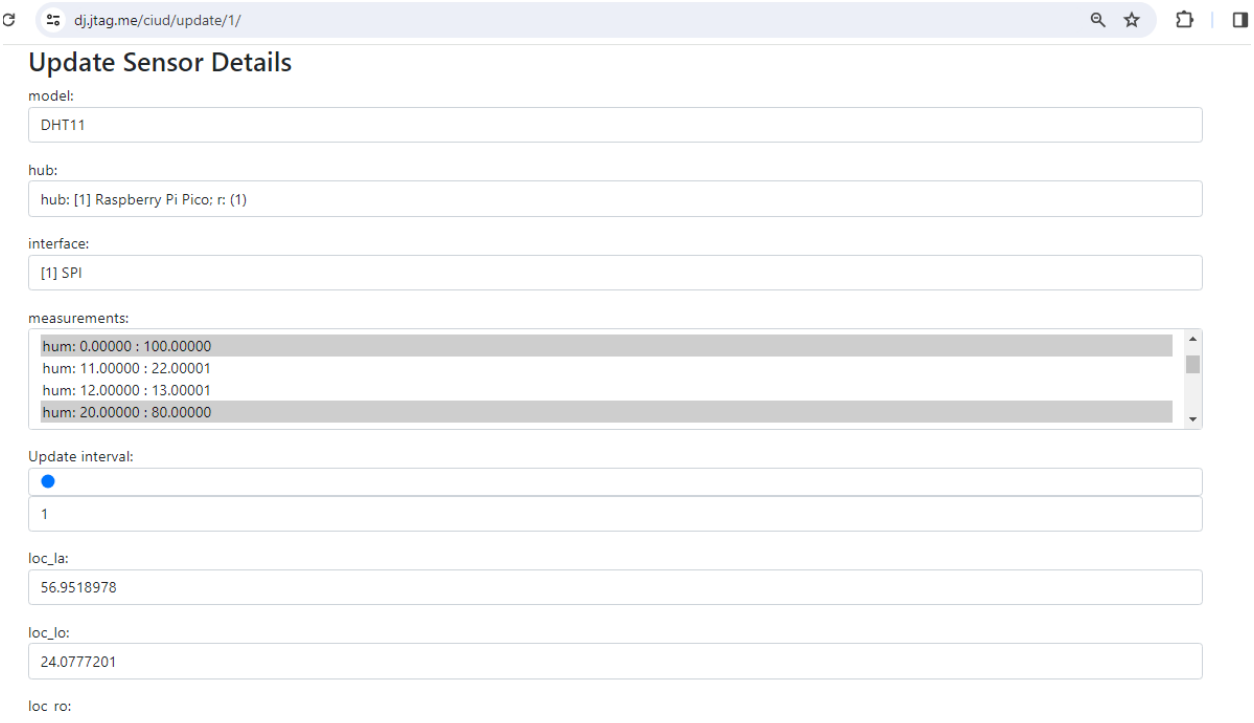
3.6.1.2 CRUD operāciju Django

Savukārt metadatu pārvaldības daļa (sensoru meklēšana, pievienošana, rediģēšana) ir izvietota 3 vietās , jo pirms prezentēšanas konstatējām ka saraksta iegūšana ļoti bremzē. Pārlikām saraksta kārtošanu un meklēšanu no backend – django – uz frontend – DataTables, taču pirmējā ielāde tāpat bija lēna. Nebija laika pētīt vai tas ir rekursīvo atslēgu hub objektam dēļ, many to many attiecībai starp sensoriem un mērapjomiem dēļ vai kā cita. Tāpēc sadalījām metadatus 3 punktos:

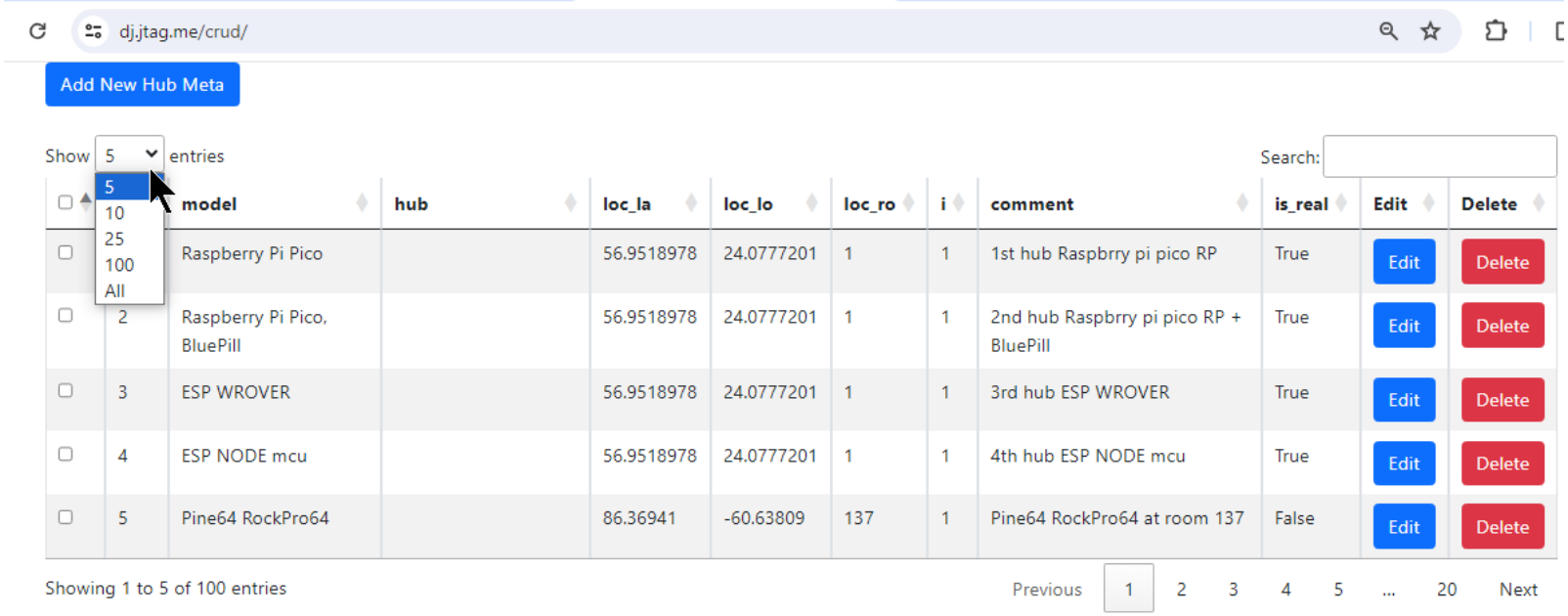
- Sensoru metadati (https://dj.jtag.me/ciud/)
- Centrmezglu metadati (https://dj.jtag.me/crud/)
- Mērapjomu metadati (https://dj.jtag.me/clud/)

Visiem trim modeļiem tiek izmantotas tikpat kā identiskas veidnes, tāpēc šeit ielikšu skatu piemērus no diviem skatiem, proti saraksta un labošanas (jo jaunizveides skats būtībā ir tas pats atjaunošanas, taču ir uzraksts add new un neaizpildīti lauki; savukārt dzēšanai nav atsevišķa skata, ir komandpoga sarakstā)

Atjaunināšanas / pievienošanas skats



Saraksta skats:



3.6.2 Mērījumu ievākšana

Mērījumu ievākšana prezentācijas brīdī norit uz 4 mikrokontrolieriem (4 atšķirīgiem modeļiem):

- Raspberry Pi Pico W
- Raspberry Pi Pico W rxtx tandēmā ar STM32F103C8T6 (iesauktu par Blue Pill)
- ESP32 WROVER CAM board
- ESP 8266MOD

Visiem kontrolieriem ir paņemta un uzbūvēta lokāli jaunākā github esošā koda micropython versija. Uz micropython main.py failā ir sarakstīti asinhroni datu ievākšanas uzdevumi un arī mērīšanas intervāla atjaunināšana ik pēc sekundes. Būtību var redzēt šī dokumenta algoritmu sadaļā.

3.6.3 Mērījumu arhivēšanas un dzēšanas uzdevums

Šis uzdevums ir rakstīts kā viena pavediena parasts python skripts, jo ir paredzēts palaišanai kā scheduled task vai chron job vai to līdzinieks. Django sistēmā meklējām ko līdzīgu C# taskiem, taču diemžēl pagaidām redzējām tikai risinājumus kas ne ar ko principiālā līmenī neatšķiras no python skripta palaišanas ar crontab. Gan celery gan django-cron, gan paša django piedāvātie risinājumi iesaista ārēju uzdevumu palaidēju, jo django ir paredzēts tikai darboties uz pieprasījuma laiku, nevis patstāvīgi būt atmiņā. Wsgi spraudņus gan neapskatījām, taču tas iekļautu vēl vienu apmācības līkni, un mazā atlikušajā laikā izvēlēties melno kasti, kad ir pieejama pietiekami labi caurredzama pašu koda alternatīva nešķita prātīgs lēmums.

Arī šī skripta būtību var apskatīt algoritmu sadaļā. Atlasa datus no datubāzes, ievieto csv un saspiež ar gzip. Tad ar pandas lasa csv un izdzēš norādīto vecumu pārsniegušos ierakstus un attiecīgi ja ir ieslēgta dzēšana no datubāzes, arī datubāzes vecos ierakstus. Laika starpība tiek glabāta kā datetime un tas ir cik attiecīgais datubāzes laukā esošais datums ir tālu no epoch (1970. gada 1. janvāra).

3.6.4 mērījumu ievākšanas saglabātāji (.py skripti uz 2 portiem),

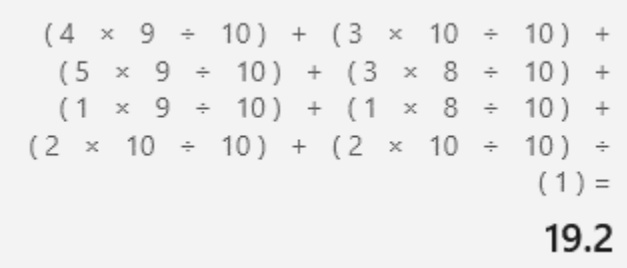
Realizēts kā asinhroni uzdevumi, temperatūra uz 911 porta, co2 uz 912 porta. Nodrošina ievākšanas intervālu sinhronizāciju. Temperatūras datu komunikācijas serviss nepārtraukti pieņem savienojumus. Tad nolasa pirmo rindu un atkarībā no tās satura izsauc attiecīgo darbību. Sinhronizācijas gadījumā dati tiek kriptēti būtībā ar plaši zināmo tīkla adrešu masku paradigmu.

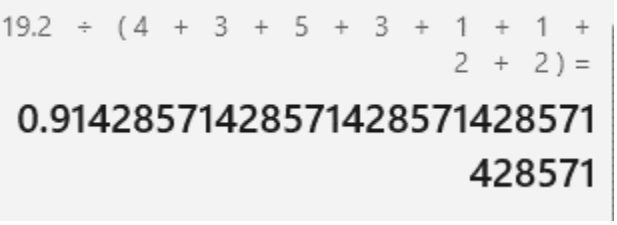
4 Novērtējums

Lai novērtētu mēs pielietojām lekcijās doto metodi, proti, piešķiram kritērijiem koeficientus, tad katru kritēriju novērtējam kādā skalā (piemēram 10 baļļu) un tad aprēķinam svērto rezultātu. (svērtais novērtējums / maksimālais novērtējums)

Novērtējot ieguvām samērā augstu novērtējuma procentu. Tas liecina ka savus mērķus sasniedzām, pat neskatoties uz to, ka, piemēram, drošība ir novērtēta augstu, jo pievienot middleware, kas tāpat kā šībrīža middleware pārraksta noklusējuma prasību uz jebkuru simbolu un garuma atļaušanu lietotājvārdā un parolē, veiktu drošības prasību nodrošināšanu, ir samērā triviāls, mazāk nekā 2 cilvēkdienās paveicams uzdevums (protams, ja drošības algoritms jau ir python valodā realizēts gatavs).

(Funkcionālie: **4***9/10 , Drošība: **3*** 10/10, Funkcijas: **5*** 9/10, Uzticamības: **3*** 8/10, Lietojamības: **1***9/10 , Efektivitātes: **1*** 8/10, Uzturamības: **2*** 10/10, Kustināmības: **2*** 10/10)/(4+3+5+3+1+1+2+2) = 19.2/21 ~= **91.43%**





Mūsu sistēmas MVP iekļauj sevī vairākus ISO25010:2011 aspektus:

Funkcionālie (9/10):

Sistēma ir labi piemērota sensoru mērījumu apskatei un pārraudzībai, kamēr vien tiek ievadīti realitātei atbilstoši metadati. Sistēmā ir iekļauti vairāki būtībā atšķirīgi sensoru tipi, tātad tā ir tuvināta reālajai situācijāi (lauka apstākļiem).

Sistēmas precizitāte ir atbilstoša konkrēto mikrokontrolieru precizitātei. Digitāliem sensoriem, protams tas ir iekšējs rādītājs, taču analogajam sensora piemēram mēs izmantojam 12 bitu ADC, kas ļauj iegūt 4096 vērtības.

Sistēma spēj sadarboties ar Raspberry pi, ja to lieto kā centrmezglu vai pat kā wifi nodrošinātāju pārējie centrmezgliem.

Drošība (10/10):

sistēmā ir iekļauts jaukšanas moments, kas neļauj ļaunprāšiem viegli paredzamā veidā iejaukties atjaunināšanas datos. Līdzīgus momentus iespējams jo viegli iekļaut arī sensoru datu pārraidē. Gadījumā ja pazūd tīkls, centrmezgli cenšas atkārtoti savienoties ar tīklu, un attiecīgi mērījumu ievākšana tiek apturēta, kad mērījumu saraksti aizņem tuvu kritiskajām atmiņas lielumus. Kad parādās tīkls, datu buferi tiek iztukšoti un mērījumi atsākti.

Funkcijas (9/10):

sasniedzām visas obligātās funkcijas, bet atmetām 3d vizualizāciju.

Sistēma atbilst tiem standartiem, kuru atbilstību ir deklarējuši attiecīgo iekārtu ražotāji. Analogajam mēram ir iekļauta rezerve gadījumiem kad strāva būtu virs 20mA, pie tam centrmezglam ir sava iekšējā aizsardzība pret īssavienojumiem.

Uzticamības (8/10): ↑

Sistēma pārbaudīti nodrošina savas funkcijas, tomēr ir samērā plašas izaugsmes iespējas.

Kā jau pieminēts, ir aizsardzība pret īssavienojumiem un rezerve augstākām strāvām. Tāpat jau minēts par tīkla darbības atjaunošanu.

Lietojamības (9/10):

Metadatu ievade ir attēlota ar pārbaudīto datatables saskarni, tāpat lietotāja grafiki ar pārbaudīto Grafana saskarni.

Mūsuprāt grafiki ir daudz pievilcīgāki par sākotnējo matplotlib versiju.

Efektivitātes (8/10):

Sistēma pēc ielādes ļauj responsīvi kārtot, šķirot metadatus. Ielādes laiku gan neizdevās uzlabot, jo pēc python3 atjauninašanas kešošanai piedāvā tikai memcached vai redis, proti ārējus risinājumus.

Mikrokontrolieru atmiņa tiek lietota maksimālā līmenī, kā redzams OUT OF MEMORY video.

Uzturamības (10/10):

Kods ir samērā viegli un mazāk kā stundas laikā saslēdzams ar jebkuru centrmezglu, kas spēj darbināt micropython, jo mainās tikai porti un varbūt LED specifika.

Kustināmības (Portability) (10/10):

Tāpat ir viegli paplašināmi metadati un pievienojami jauni objekti pēc nepieciešamības.

5 Secinājumi

Mums izdevās šajā īsajā laikā radīt samērā dzīvotspējīgu MVP Universitātes telpu gaisa kvalitātes pārraudzības sistēmu, kuru pateicoties tehnoloģiju izvēlei ir ļoti viegli mērogot vairāku kvalitātes kritēriju jomā, piemēram, drošības, uzticamības, kvalitātes.